Отчет по лабораторной работе 8

Граф хранится в CSR (offsets, edges); вход — файл «N M» и M пар «u v», каждый MPI-процесс читает его сам, для неориентированного графа ребро добавляется в обе стороны (directed=1 — ориентированный). Чтение и CSR (0.02–11 с) в замер не входят. Графы с фиксированным seed: ER — M случайных рёбер, R-MAT (a, b, c = 0.57, 0.19, 0.19, степенное распределение, макс. степень 66 041), решётка 500 × 500. Последовательный BFS — очередь. Гибридный — по уровням: все процессы хранят одинаковые frontier и dist; процесс rank берёт индексы фронта rank, rank + P, …, его T потоков обходят их через omp for schedule(runtime) (по умолчанию dynamic, 64) и пишут найденные u с dist[u] = −1 в свои динамические массивы (выровнены на 64 Б — без false sharing). Гонок нет: в цикле dist только читается, общий счётчик и next_frontier не трогаются. После цикла потоки ставят local_candidate[u] = 1, MPI_Allreduce(MPI_MAX) даёт всем global_candidate, и каждый процесс одинаково строит next_frontier по возрастанию u с проверкой dist[u] = −1, так что повторные находки отбрасываются. Время: MPI_Barrier, MPI_Wtime, MPI_Reduce(MPI_MAX); медиана 5 запусков после прогрева (и медиана повторных запусков одной конфигурации). $S = T_{seq} / T_{par}$, $E = S / (P \cdot T)$.

Запуск: mpicc -O2 -fopenmp bfs.c -o bfs; OMP_NUM_THREADS=3 mpiexec --bind-to none -n 2 ./bfs graph.txt 0 [schedule=static|dynamic chunk=K] [exchange=allgatherv] [stats=1]. Система: i5-9400F (6 ядер), WSL2 Ubuntu 26.04, Open MPI 5.0.10, gcc 15.2; * — больше 6 потоков, переподписка.

3.1 + 3.5. Корректность (dist сравнивается поэлементно, на всех процессах dist одинаков) и граф

Граф	|V|	|E|	d	s	Уровни	Недостиж.	Несовп.	T_{seq}, с	T 2×3, с	S 2×3	T 1×6, с	S 1×6
ER 250K	250000	2000000	16.0	0	7	0	0	0.0226	0.0085	2.66	0.0055	4.12
ER 1M, d=4	1000000	2000000	4.0	0	17	19728	0	0.0926	0.0629	1.47	0.0349	2.65
ER 1M, d=16	1000000	8000000	16.0	0	8	2	0	0.1684	0.0546	3.09	0.0335	5.03
ER 1M, d=64	1000000	32000000	64.0	0	5	0	0	0.3846	0.0912	4.22	0.0570	6.75
ER 4M, d=16	4000000	32000000	16.0	0	9	0	0	1.2845	0.3710	3.46	0.2421	5.31
R-MAT 2^{20}	1048576	8000000	15.3	64098	6	509608	0	0.0822	0.0412	2.00	0.0273	3.01
решётка 500^2	250000	499000	4.0	0	999	0	0	0.0021	0.3484	0.01	0.1797	0.01

3.2. Число процессов, T = 2

P	T	P·T	Время, с	S	E	Обмен,%
1	2	2	0.0636	2.65	1.32	3
2	2	4	0.0679	2.48	0.62	20
3	2	6	0.0875	1.92	0.32	34
4	2	8*	0.1189	1.42	0.18	44
8	2	16*	0.2163	0.78	0.05	62

3.3. Число потоков, P = 2

P	T	P·T	Время, с	S	E	Обмен,%
2	1	2	0.1016	1.66	0.83	15
2	2	4	0.0679	2.48	0.62	20
2	3	6	0.0546	3.09	0.51	24
2	4	8*	0.0791	2.13	0.27	30
2	8	16*	0.0601	2.80	0.17	23

3.4. Одинаковое P·T (6 и 16)

P	T	P·T	Время, с	S	E	Обмен,%
1	6	6	0.0335	5.03	0.84	6
2	3	6	0.0546	3.09	0.51	24
3	2	6	0.0875	1.92	0.32	34
6	1	6	0.1514	1.11	0.19	43
1	16	16*	0.0422	3.99	0.25	5
2	8	16*	0.0601	2.80	0.17	23
4	4	16*	0.2269	0.74	0.05	31
8	2	16*	0.2163	0.78	0.05	62
16	1	16*	0.2820	0.60	0.04	73

3.6. Фронты, балансировка и планировки OpenMP (время, мс; † — P = 2)

Граф	Размеры фронтов по уровням	Макс. фронт	Рёбра max/ср (P=4)	2×3 static	dyn,1	dyn,64	1×6 static	dyn,1	dyn,64
ER 1M, d=16	1 → 18 → 284 → 4508 → 68996 → 618986 → 307189 → 16	618986	1.000	54	71	55	33	56	40
R-MAT 2^{20}	1 → 38328 → 437879 → 62211 → 543 → 6	437879	1.018	43	51	41	28	39	27
решётка 500^2	1 → 2 → … → 500 → … → 1	500	1.001†	353	356	348	—	—	—

3.7. Плотный массив (MPI_Allreduce, |V| int на уровень) и списки вершин (доп. задание, MPI_Allgatherv)

Граф	2×3: Allreduce, T / обмен, с	2×3: Allgatherv	6×1: Allreduce	6×1: Allgatherv
ER 1M, d=16	0.0546 / 0.0134	0.0719 / 0.0018	0.1514 / 0.0652	0.1581 / 0.0111
ER 4M, d=16	0.3710 / 0.0619	0.5132 / 0.0098	0.6577 / 0.2859	0.7713 / 0.0636
R-MAT 2^{20}	0.0412 / 0.0105	0.0937 / 0.0109	0.1139 / 0.0542	0.0974 / 0.0098
решётка 500^2	0.3484 / 0.1997	0.0105 / 0.0020	1.4338 / 1.2201	0.0096 / 0.0048

Вывод:

1) Расстояния гибридного и последовательного BFS совпали для всех 7 графов и всех P, T, планировок и способов обмена. 2) На одной 6-ядерной машине потоки выгоднее процессов: при 6 потоках 1×6 даёт S = 5.0, 2×3 — 3.1, 3×2 — 1.9, 6×1 — 1.1. Каждый лишний процесс добавляет Allreduce 4 МБ на уровень (доля обмена 6 → 44 %) и повторяет у себя работу O(|V|) на уровне (обнуление признаков, просмотр global_candidate), которая не делится. Поэтому одинаковое P·T не даёт одинакового времени. 3) При переподписке libgomp ждёт активно (spin), если T ≤ 6, и процессы отнимают друг у друга ядра: 2×4 и 4×4 медленнее 2×8 и 1×16. С OMP_WAIT_POLICY=passive 2×4 ускоряется с 0.077 до 0.055 с, 4×4 — с 0.233 до 0.113 с. 4) Чем больше степень, тем больше работы на уровень относительно накладных O(|V|): S(1×6) = 2.6 при d = 4, 5.0 при d = 16, 6.7 при d = 64. E > 1 при малом числе потоков: следующий фронт идёт по возрастанию номеров вершин, и обход CSR локальнее, чем по очереди. Решётка — худший случай: 999 уровней с фронтом ≤ 500, каждый стоит O(|V|) + Allreduce, и BFS в 85 –170 раз медленнее последовательного. 5) Работа сосредоточена в 2–3 уровнях (ER: 69 тыс. → 619 тыс. → 307 тыс. вершин); на первых и последних уровнях фронт меньше P·T, и часть потоков простаивает. Циклическая раздача даёт процессам поровну вершин (±2), а по рёбрам дисбаланс 1.000 у ER и 1.018 у R-MAT: у R-MAT уровень 0 — одна вершина степени 66 041, её обходит один поток одного процесса. dynamic,1 в 1.2–1.7 раза медленнее (кроме решётки) из-за раздачи по одной вершине; static и dynamic,64 равны в пределах разброса запусков (±20 %). 6) Allreduce передаёт |V| признаков на каждом уровне, даже если новых вершин мало: ≈ L·4|V| байт (ER 1M — 32 МБ, решётка — 1 ГБ). Это избыточно и заметно при многих уровнях и процессах (решётка 6×1 — 85 % времени). Списки через Allgatherv сокращают время обмена в 5–7 раз на ER и в 100–250 раз на решётке (на R-MAT 2×3 списки почти размером |V|, выигрыша нет); решётка ускоряется в 33–150 раз. Но на ER и R-MAT общее время выросло: удаление повторов из списков потоков идёт последовательно, а фронт теряет упорядоченность. 7) Копия графа в каждом процессе (ER 4M: CSR 272 МБ + 7 массивов по |V|, ≈ 380 МБ на процесс) ограничивает размер графа памятью одного узла. Для масштабирования граф нужно делить между процессами: процесс хранит свои вершины и пересылает найденных соседей их владельцам.